TALLER: B1-T1

Toolkit de Extracción, Análisis Bursátil y Gestión de Carteras

Datos básicos:

- Grupos de 3 estudiantes.
- Entrega 18:00 del día 12 de Noviembre, antes de la clase de evaluación.
- El entregable es un github a través del aula virtual. El día de la evaluación se realizará una presentación de 5 minutos.

1. Objetivo de la práctica

El objetivo de este proyecto es crear una toolbox en Python para la obtención, estandarización y análisis de información bursátil, así como la gestión y simulación de carteras de inversión.

Si bien una parte importante del código puede resolverse con el apoyo de herramientas de IA, la relevancia pedagógica radica en asimilar en profundidad las buenas prácticas y patrones de diseño en ingeniería de software: toma de decisiones sobre abstracciones, modelado orientado a objetos mediante DataClasses, modularidad, diseño desacoplado y estandarización de formatos de datos. Estas decisiones son las que permiten a los proyectos analíticos y cuantitativos escalar más allá de los límites de un desarrollo individual.

2. Contexto del problema

En el ámbito financiero y cuantitativo, es común desarrollar scripts rápidos para descargar series de precios y calcular métricas básicas. Sin embargo, cuando se requiere combinar distintas fuentes de información (diferentes APIs de mercado) o gestionar activos de diversa naturaleza, la falta de una arquitectura estructurada genera fragilidad y deuda técnica.

El reto consiste en construir un sistema modular donde la capa de ingesta e integración de APIs esté desacoplada de la capa de análisis, modelado de series y simulación de carteras, permitiendo añadir nuevas fuentes o tipos de análisis sin alterar el núcleo del sistema.

3. Fuentes de información y entorno técnico

Fuentes de datos (APIs en línea): Integración con servicios de datos de mercado con acceso programático (por ejemplo: Yahoo Finance / yfinance, Alpha Vantage, Finnhub, Polygon, etc.).

- Tipología de series requeridas:
    - Precios históricos de acciones individuales (equities).
    - Precios históricos de índices bursátiles (por ejemplo: S&P 500, IBEX 35, Euro Stoxx 50).
    - Tipología de datos adicional a criterio del estudiante (fundamentales contables, indicadores macroeconómicos, tipos de interés, materias primas o métricas de sentimiento).
- Estandarización de salida: Independientemente de la API consultada, la salida del extractor debe normalizarse en objetos internos homogéneos e interoperables.

4. Tarea del estudiante

El estudiante deberá estructurar y programar el paquete de software resolviendo los siguientes bloques:

4.1. Repositorio en GitHub y despliegue Plug-and-Play

- Documentación (README.md): Redactar un fichero README detallado con la descripción del toolkit, prerrequisitos, instrucciones claras paso a paso para instalación y uso, y ejemplos de ejecución reproducibles.
- Estructura en /src: Organizar el núcleo del proyecto y todos los módulos fuente dentro del directorio /src.
- Configuración plug-and-play: Añadir los archivos de gestión de dependencias (requirements.txt o pyproject.toml), gitignore y cualquier elemento necesario para que el proyecto pueda ser clonado y ejecutado directamente sin fricciones de entorno.

4.2. Ingesta y extractor multifuente de datos

- Descarga de acciones e índices: Implementar métodos para consultar y descargar información histórica de precios de acciones e índices bursátiles.
- Estandarización de salidas: Garantizar que, con independencia de la API utilizada, el programa devuelva objetos idénticos y compatibles entre sí.
- Tipología de datos adicional: Incorporar al menos una fuente o tipo de datos adicional a elección del grupo (macroeconomía, fundamentales, balances, etc.).
- Descarga simultánea de N series: Habilitar la extracción de múltiples series temporales de manera simultánea o en lote a partir de un input compuesto (por ejemplo, una lista de activos).

4.3. Modelado: Series de precios y Cartera (2.0 puntos)

- DataClasses para series de precios Crear DataClasses que representen de manera coherente cada serie temporal de precios con sus metadatos y valores.
- Conceptualización y modelado de Cartera. Definir conceptualmente y programar la clase correspondiente a una cartera de inversión, gestionando de forma adecuada la agregación de múltiples activos y sus ponderaciones.
- Estadísticos automáticos: Incorporar métodos analíticos a las DataClasses de precios, asegurando que los estadísticos fundamentales (media y desviación típica) se apliquen de forma automática al instanciar el objeto.

4.4. Motor de simulación estocástica de Monte Carlo

- Simulación parametrizable: Diseñar un programa de simulación de Monte Carlo maleable por parámetros (número de iteraciones, horizonte temporal, supuestos de volatilidad y retorno), aplicable tanto a un activo individual como a la cartera global.
- Integración como método en Cartera: Incorporar el algoritmo de Monte Carlo como un método directo dentro de la DataClass de la cartera.
- Visualización del resultado: Implementar un método para mostrar por pantalla visualmente las trayectorias y distribuciones resultantes de la simulación.

4.5. Limpieza, preprocesado y reportes

- Limpieza y preprocesado: Desarrollar rutinas para la limpieza de datos (gestión de valores nulos o NaNs, alineación temporal entre activos con distintos calendarios bursátiles, etc.).
- Validación de inputs: Analizar y justificar si el programa debería admitir cualquier tipo de input siempre que contenga una serie temporal de precios o si conviene establecer restricciones más estrictas.
- Método .report() en Markdown: Implementar en la cartera un método .report() parametrizable que genere texto estructurado en Markdown con métricas clave, diagnósticos y advertencias de riesgo.
- Método .plots_report(): Diseñar un conjunto de visualizaciones analíticas de valor añadido (comparativas de rendimiento, matrices de correlación, evolución de cartera) accesibles mediante el método .plots_report().

4.6. Estructura de proyecto y diagramado

- Representar la jerarquía de clases, dependencias y flujo del sistema mediante un diagrama formal de arquitectura, utilizando recursos como FossFLOW o diagramas UML equivalentes.

5. Entregables

Repositorio de GitHub: Enlace al repositorio con el código fuente y el historial de commits.

Presentación. Cada grupo de estudiantes tiene 5 minutos para enseñar y explicar su práctica.

6. Criterios de evaluación

Los indicados arriba para la parte del github. 50% de la nota Github + 50% presentación.